

TigerCS ↔ Genesys Integration Guide

Developer guide for the Genesys engineering team: authentication, endpoints, payloads, channel flows, agent mapping, human handoff and UAT scenarios.

Document	TigerCS-Genesys Integration Guide
Version	1.0 — for UAT
Date	24 September 2026
Audience	Genesys technical team; Tiger IT
Verified against	TigerCS source code, <code>main</code> branch (commit <code>16b0dad</code>). Every endpoint, field name, status code and rule in this guide was checked against the controllers, contracts, services, migrations and tests in that commit.
Classification	Confidential. Contains no credentials, keys or secrets. Every value shown in angle brackets (for example <code><TIGERCS_BASE_URL></code>) is a placeholder.

Contents

1. Integration Overview
2. Architecture
3. Authentication
4. API Quick Reference
5. Phone Call Flow
6. Digital Channel Flow
7. Customer Lookup
8. Create Ticket
9. Unclassified Tickets and SLA
10. Update Ticket / Conversation
11. Transcript Handling
12. Genesys Agent Mapping and Interaction Ownership
13. Human Handoff
14. Error Handling
15. UAT Testing Scenarios
16. Information Required from Genesys
17. Security Notes
18. End-to-End Flow Summary

Key facts at a glance

Topic	Current implementation
Base path	<TIGERCS_BASE_URL>/api/genesys/...
Authentication	Authorization: Bearer <JWT> obtained from POST /api/auth/login using a TigerCS service account. No API-key scheme exists in the current code (see §3).
Payload format	JSON (application/json). Response properties are camelCase. Request property names are case-insensitive.
Idempotency key	conversationId : one Genesys conversation always maps to exactly one TigerCS ticket.
Ringing calls	Never sent to TigerCS. A phone ticket is created at agent pickup .
Ticket closure	Never caused by Genesys. Ending a conversation does not close, resolve or re-status a ticket.
Feature switch	Genesys:Enabled (server setting). When it is false, every Genesys endpoint returns 503 .

IMPLEMENTATION GAP — Authentication differs from the target architecture

The target architecture keeps the TigerCS username and password out of Genesys: a Tiger Group integration layer would hold them server-side, obtain and reuse the JWT, and expose its own credential (for example an API key) to Genesys.

That integration layer is not part of the current TigerCS codebase. Today the Genesys endpoints accept only a TigerCS JWT. Section 3 describes both the current behaviour and the target, and lists the decision Tiger IT must take before go-live.

1. TigerCS - Genesys Integration Overview

Genesys and TigerCS each own a distinct part of the customer journey. The integration is deliberately narrow: Genesys tells TigerCS **conversation facts**, and TigerCS owns **everything about the ticket**.

Genesys owns	TigerCS owns
Channels: phone, website chat, chatbot / virtual agent, WhatsApp, social media	Tickets and the ticket lifecycle (status, resolution, closure, reopen)
Routing, queues, IVR	Customer context: CRM buyer, units, PACT and Tasleeh matches, ticket history
Conversations and their lifecycle	Interaction history and stored transcripts
Agent availability, presence and transfers	Agent mapping and interaction ownership ("Handled By")
Communication delivery (calling, chat, WhatsApp and social messaging)	Classification, workflow, approvals, SLA and escalation
Deciding when a human is needed	Recording pending human work and showing it to TigerCS agents

Guiding rules

- **One conversation, one ticket.** Every channel uses the same Create endpoint. Retries never create duplicates.
- **TigerCS executes nothing on a channel.** It does not dial, send chat messages, or send WhatsApp or social replies. Genesys delivers all communication.
- **Genesys never reaches Tiger CRM, PACT or Tasleeh directly.** Customer lookup runs inside TigerCS.
- **Genesys cannot change ticket business fields.** Status, category, request type, priority, owner, department, resolution and closure are not in the contract.

2. Architecture

2.1 Target architecture



Figure 1 — Target: Genesys authenticates to the integration layer; the layer authenticates to TigerCS with server-side credentials.

2.2 Current implementation (verified in code)



Figure 2 — Current: the caller logs in to TigerCS as a service account and calls the Genesys endpoints directly with a JWT.

Aspect	Current code	Target architecture
Who calls <code>/api/genesys/*</code>	Any client holding a valid TigerCS JWT	Only the Tiger Group integration layer
What Genesys authenticates with	TigerCS service-account JWT (<code>Authorization: Bearer</code>)	A credential issued by the integration layer (header name to be confirmed when it is built)
Where the TigerCS username/password live	With whichever component calls <code>POST /api/auth/login</code>	Server-side only, in integration-layer settings (e.g. <code>Ticketing__Username</code> , <code>Ticketing__Password</code>)
JWT handling	Caller logs in, caches the token, and re-logs in before expiry	Integration layer obtains, caches and reuses the token; Genesys never sees it
TigerCS API contract	As documented in this guide	Unchanged. The layer forwards the same requests and responses

The endpoints, payloads and business rules in sections 4–15 are the same in both architectures. Only the authentication hop changes.

3. Authentication

3.1 Current implementation

Every Genesys endpoint sits behind `[Authorize(Policy = "CustomerVerification")]` and standard ASP.NET JWT bearer authentication. No webhook signature, HMAC, OAuth client-credentials grant or API-key header is implemented for Genesys.

Requirement	Detail
Header on every Genesys call	<code>Authorization: Bearer <ACCESS_TOKEN></code>
Content type	<code>Content-Type: application/json</code> on POST and PATCH
How to obtain the token	<code>POST /api/auth/login</code> with a dedicated TigerCS service account
Required role	The service account must hold CS Agent or CS Supervisor . (System Administrator also passes through a central override. Do not use an administrator account for the integration.)
Token lifetime	Server setting <code>Jwt:ExpirationMinutes</code> ; the default is 60 minutes . The login response returns <code>expiresAtUtc</code> . Re-login shortly before expiry, or once on a <code>401</code> .
Missing or expired token	<code>401 Unauthorized</code>
Valid token, wrong role	<code>403 Forbidden</code>

Login request (placeholders only)

```
POST <TIGERCS_BASE_URL>/api/auth/login
Content-Type: application/json
```

```
{ "username": "<SERVICE_ACCOUNT_USERNAME>", "password": "<SERVICE_ACCOUNT_PASSWORD>" }
```

Login response

```
200 OK
{
  "accessToken": "<JWT>",
  "expiresAtUtc": "2026-09-24T10:15:00Z",
  "employeeId": "<GUID>",
  "displayName": "<service account display name>",
  "roles": [ "CS Agent" ],
  "primaryDepartmentId": 3
}
```

Login can also return `400` (malformed body), `401` (invalid credentials) and `423 Locked` (account locked out after repeated failures). Do not retry a `401` or `423` from login in a loop: it can lock the service account.

3.2 What stays server-side

- The JWT signing key and issuer/audience settings (`Jwt:*`).
- Credentials for Tiger CRM, PACT (`PactApi:ApiKey`) and Tasleeh. Genesys never calls these systems.

- Database connection strings and all other TigerCS configuration.
- In the target architecture, the TigerCS service-account username and password (`Ticketing__Username`, `Ticketing__Password`).

3.3 Target architecture and open decision

IMPLEMENTATION GAP — Genesys currently needs the service-account credentials

Because the integration layer does not exist in the current code, whichever component calls `/api/auth/login` must hold the TigerCS service-account username and password. That conflicts with the rule that Genesys must not receive them.

Tiger IT decision required before go-live: (a) build and deploy the Tiger Group Genesys Integration Layer, which holds the credentials server-side and exposes its own credential to Genesys; or (b) as an interim measure, allow the credentials to be stored **only** in the Genesys integration's secure credential store (never in flows, scripts or logs).

This guide will be reissued with the exact header and endpoint base URL once the integration layer is implemented. Until then, do not assume an `X-API-Key` header exists: TigerCS will reject such requests with `401`.

4. API Quick Reference

These are the only Genesys-facing endpoints in the current code. There is no webhook endpoint, no "failed interactions" endpoint and no per-channel endpoint. Any such endpoint in older design documents is obsolete.

Method	Path	Purpose	Genesys calls it when
GET	<code>/api/genesys/customers/lookup</code>	Customer, units and existing tickets for a phone number	Phone: immediately after agent pickup. Optional for digital channels.
POST	<code>/api/genesys/tickets</code>	Create, or reuse, the one ticket for a conversation	Phone: after pickup (after lookup). Digital: when the conversation starts.
PATCH	<code>/api/genesys/tickets/{ticketId}</code>	Agent info, conversation end, transcript, human handoff	Whenever these facts change, and always when the conversation ends.
POST	<code>/api/genesys/agent-context</code>	Strictly resolve a Genesys agent to a TigerCS user and record them as the interaction handler	Optional. When an agent opens or works TigerCS from Genesys.

All four endpoints need the authentication in §3 and return `503` when `Genesys:Enabled` is false. Error bodies use the standard RFC 7807 ProblemDetails format (§14).

4.1 Customer Lookup

GET `/api/genesys/customers/lookup?phoneNumber={phoneNumber}`

When	Phone: right after the agent picks up, before Create Ticket. Digital: optional, when a phone number is known.
Required	<code>phoneNumber</code> (query string). The caller's number as Genesys received it.
Optional	None.
Success	<code>200 OK</code> , including when no customer is found (<code>found: false</code>).
Errors	<code>400</code> <code>phoneNumber</code> missing or blank · <code>401</code> · <code>403</code> · <code>503</code> integration disabled.
Retry / idempotency	Read-only; nothing is created. Safe to retry. Never block the call flow on it.

Request

```
GET <TIGERCS_BASE_URL>/api/genesys/customers/lookup?phoneNumber=%2B9627XXXXXXX
Authorization: Bearer <ACCESS_TOKEN>
```

Response (values illustrative)

```
200 OK
{
  "phoneNumber": "+9627XXXXXXX",
  "found": true,
```

```

"crmStatus": "Found",
"crmBuyers": [
  {
    "customer": { "customerId": 1001, "fullNameEnglish": "<name>",
                  "fullNameArabic": "<name>", "mobileNumber": "+9627XXXXXXX",
                  "email": "<email>" },
    "units": [
      { "leadId": 501, "leadStatus": 3, "leadStatusName": "<status>", "unitId": 7001,
        "unitNumber": "A-101", "unitStatus": 2, "unitType": 1, "floorNumber": 1,
        "projectId": 12, "projectName": "<project>", "projectArabicName": "<project>",
        "customerType": 1, "customerTypeName": "<type>" }
    ]
  }
],
"externalSources": [
  { "source": "Pact", "status": "NotFound", "customers": [] },
  { "source": "Tasleeh", "status": "Found", "customers": [
    { "externalCustomerId": "<id>", "displayName": "<name>",
      "phoneNumber": "+9627XXXXXXX", "email": null, "customerType": "<type>",
      "units": [
        { "externalUnitId": "<id>", "unitNumber": "B-204",
          "propertyName": "<property>", "towerName": "<tower>",
          "unitType": "<type>", "unitReferenceId": null,
          "contactReferenceId": null } ] } ] }
],
"tickets": [
  { "ticketId": 1234, "ticketNumber": "<ticketNumber>",
    "ticketStatus": "InProgress", "isOpen": true, "requestSummary": "<summary>",
    "currentDepartmentId": 3, "createdAtUtc": "2026-09-20T08:11:00Z" }
],
"openTicketCount": 1
}

```

4.2 Create / Reuse Ticket

POST /api/genesys/tickets

When	Phone: after agent pickup (after Customer Lookup). Digital (website chat, chatbot, WhatsApp, social): when the conversation starts.
Required	<code>conversationId</code> (max 64 chars); <code>channel</code> : one of <code>Phone</code> , <code>WebsiteChat</code> , <code>WhatsApp</code> , <code>SocialMedia</code> (case-insensitive).
Required for routing	At least one of <code>departmentId</code> , <code>departmentCode</code> or a mapped <code>queueId</code> . Without one, the call fails with 422 (§8.3).
Optional	<code>customerPhone</code> , <code>customerName</code> , <code>customerEmail</code> , <code>calledNumber</code> , <code>queueId</code> , <code>queueName</code> , <code>agentId</code> , <code>agentName</code> , <code>startedAtUtc</code> , <code>direction</code> , <code>departmentId</code> , <code>departmentCode</code> , <code>towerName</code> , <code>unitNumber</code> , <code>subject</code> , <code>interactionId</code> , <code>participantId</code> , <code>communicationId</code> .
Success	201 Created with outcome <code>TicketCreated</code> (and a <code>Location</code> header), or 200 OK with outcome <code>AlreadyIngested</code> .
Errors	400 unsupported channel, missing or blank <code>conversationId</code> , or malformed JSON · 401 · 403 · 422 department not resolved, channel not configured, or ticket creation refused · 503.

Retry / idempotency	Idempotent on <code>conversationId</code> . Retrying returns the same ticket. Safe under concurrent delivery (unique index).
----------------------------	--

Field	Type	Notes
<code>conversationId</code>	string ≤64	Required. Genesys conversation ID; the idempotency key.
<code>channel</code>	string	Required. <code>Phone</code> · <code>WebsiteChat</code> · <code>WhatsApp</code> · <code>SocialMedia</code> . A chatbot on the website uses <code>WebsiteChat</code> .
<code>customerPhone</code>	string	Caller or customer mobile number. Used for the internal customer lookup. May be omitted (withheld caller ID, social DM).
<code>customerName</code> , <code>customerEmail</code>	string	As collected by the channel (e.g. website chat form).
<code>calledNumber</code>	string	Tiger number dialled. Recorded only; not used for routing.
<code>queueId</code>	string ≤64	Stable Genesys queue ID. Resolved to a department through the queue mapping.
<code>queueName</code>	string	Display only.
<code>agentId</code>	string ≤64	The handling agent's immutable Genesys User ID (not a name). Resolved server-side (§12).
<code>agentName</code>	string	Display only; never used as an identity key.
<code>startedAtUtc</code>	ISO-8601 UTC	When the interaction started.
<code>direction</code>	string	<code>Inbound</code> / <code>Outbound</code> where available.
<code>departmentId</code> / <code>departmentCode</code>	int / string	The department the customer explicitly chose (e.g. website form). Takes priority over the queue mapping.
<code>towerName</code> , <code>unitNumber</code>	string	Typed by the customer; stored as an unverified manual snapshot.
<code>subject</code>	string	Becomes the ticket's request summary. If omitted, a factual channel text is used (e.g. "Phone call received via Genesys").
<code>interactionId</code> , <code>participantId</code> , <code>communicationId</code>	string	Accepted for forward compatibility. Not currently persisted.

Request — phone, after pickup

```
POST <TIGERCS_BASE_URL>/api/genesys/tickets
Authorization: Bearer <ACCESS_TOKEN>
Content-Type: application/json
```

```
{
  "conversationId": "<GENESYS_CONVERSATION_ID>",
  "channel": "Phone",
  "direction": "Inbound",
  "customerPhone": "+9627XXXXXXX",
  "calledNumber": "<TIGER_HOTLINE>",
```

```

"queueId": "<GENESYS_QUEUE_ID>",
"queueName": "Customer Service",
"agentId": "<GENESYS_USER_ID>",
"agentName": "<Agent display name>",
"startedAtUtc": "2026-09-24T09:15:00Z"
}

```

Responses

```

201 Created
Location: /api/tickets/1234
{ "outcome": "TicketCreated", "conversationId": "<GENESYS_CONVERSATION_ID>",
  "ticketId": 1234, "ticketNumber": "<ticketNumber>" }

200 OK   (same conversationId sent again)
{ "outcome": "AlreadyIngested", "conversationId": "<GENESYS_CONVERSATION_ID>",
  "ticketId": 1234, "ticketNumber": "<ticketNumber>" }

```

Store the returned **ticketId**. It is the route parameter for every later update.

4.3 Update Ticket / Conversation

PATCH /api/genesys/tickets/{ticketId}

When	When the agent becomes known, when a human is needed or takes over, and always when the conversation ends or disconnects.
Required	conversationId , which must belong to the ticket in the route. ticketId in the path.
Optional	agentId , agentName , ended { endedAtUtc , endReason , transcript[] }, handoff { required , agentAvailable , mode , reason , trigger , workItemId , assignedAgentId }. Send only the parts that changed.
Success	200 OK with outcome Applied .
Errors	400 blank conversationId, invalid transcript or unknown handoff mode · 401 · 403 · 404 ticket not found or conversation never ingested · 409 conversation belongs to another ticket · 422 assignedAgentId sent with no outstanding human work · 503. See §14 for a known 500 case.
Retry / idempotency	Every part is independently idempotent: resending is safe (§10.4, §11.3, §13.5).

Request — conversation ended with transcript

```

PATCH <TIGERCS_BASE_URL>/api/genesys/tickets/1234
Authorization: Bearer <ACCESS_TOKEN>
Content-Type: application/json

{
  "conversationId": "<GENESYS_CONVERSATION_ID>",
  "agentId": "<GENESYS_USER_ID>",
  "agentName": "<Agent display name>",
  "ended": {
    "endedAtUtc": "2026-09-24T09:31:12Z",

```

```

    "endReason": "CustomerDisconnect",
    "transcript": [
      { "externalMessageId": "<MSG_ID_1>", "sender": "Customer",
        "senderId": "<ID>", "senderName": "<name>",
        "sentAtUtc": "2026-09-24T09:15:03Z", "body": "Hello, I need an NOC." },
      { "externalMessageId": "<MSG_ID_2>", "sender": "VirtualAgent",
        "senderName": "Tiger Bot",
        "sentAtUtc": "2026-09-24T09:15:05Z", "body": "Is the NOC for a resale?" }
    ]
  }
}

```

Response

```

200 OK
{
  "outcome": "Applied",
  "conversationId": "<GENESYS_CONVERSATION_ID>",
  "ticketId": 1234,
  "ticketNumber": "<ticketNumber>",
  "ticketStatus": "Open",
  "conversationEnded": true,
  "transcriptMessageCount": 2,
  "handoffStatus": null,
  "ticketAgentHandoffId": null
}

```

4.4 Agent Context (optional, agent action)

POST /api/genesys/agent-context

When	Optional. When an agent opens or works TigerCS from Genesys and the integration needs to know which TigerCS user that agent is, or to record the handler strictly.
Required	genesysUserId : the agent's immutable Genesys User ID (max 64 chars).
Optional	conversationId : when supplied, that interaction records the resolved user as its handler. agentEmail : audit only, never used for matching.
Success	200 OK with outcome Resolved .
Errors	400 genesysUserId missing, blank or too long · 401 · 403 with code GENESYS_AGENT_NOT_MAPPED or GENESYS_AGENT_INACTIVE (or a plain 403 if the service account lacks the role) · 404 conversation never ingested · 503.
Retry / idempotency	Safe to repeat. The first resolved handler of an interaction is kept; later calls never overwrite it.

Request and response

```

POST <TIGERCS_BASE_URL>/api/genesys/agent-context
{ "genesysUserId": "<GENESYS_USER_ID>", "agentEmail": "<agent email>",
  "conversationId": "<GENESYS_CONVERSATION_ID>" }

```

```
200 OK
{
  "outcome": "Resolved",
  "genesysUserId": "<GENESYS_USER_ID>",
  "userId": "<TIGERCS_USER_GUID>",
  "userName": "<tigercs username>",
  "displayName": "<display name>",
  "roles": [ "CS Agent" ],
  "departmentIds": [ 3 ],
  "conversationId": "<GENESYS_CONVERSATION_ID>",
  "ticketId": 1234,
  "ticketNumber": "<ticketNumber>",
  "ticketInteractionId": 5678,
  "handledByUserId": "<TIGERCS_USER_GUID>"
}
```

5. Phone Call Flow

RULE — Ringing must not create a ticket

TigerCS has no "ringing" or call-progress event. A call that rings, is abandoned, or ends in the IVR never reaches TigerCS and never produces a ticket.



Figure 3 — Phone call flow. The dashed box is Genesys-only; no TigerCS call is made.

#	Genesys event	TigerCS call	Notes
1	Call rings, IVR, queueing	None	Do not call TigerCS.
2	Agent answers (pickup)	GET /api/genesys/customers/lookup?phoneNumber=	Shows the agent who is calling and any open tickets. <code>found:false</code> is normal.
3	Immediately after step 2	POST /api/genesys/tickets with <code>channel:"Phone", customerPhone, queueId, agentId, agentName</code>	Creates or reuses the ticket. Store <code>ticketId</code> . Do not wait for the lookup to succeed.
4	Agent known or changed	PATCH .../tickets/{ticketId} with <code>agentId, agentName</code>	Only needed if the agent was not in step 3. The first agent recorded is kept.
5	Call ends (any party, any reason)	PATCH .../tickets/{ticketId} with <code>ended { endedAtUtc, endReason }</code>	Transcript optional for voice. The ticket stays open.

Why pickup is the trigger

- **No empty tickets.** Abandoned, missed and IVR-only calls never create work that nobody requested.
- **The agent is known.** At pickup Genesys can send the Genesys User ID, so TigerCS records who handled the interaction from the first call.
- **Context arrives when it is useful.** The lookup result (customer, units, open tickets) is ready just as the agent starts speaking.
- **Matches the TigerCS design.** The Create endpoint deliberately has no event field; posting means "a real inquiry exists".

NOTE — End events for unanswered calls

If Genesys sends an update for a conversation that never reached pickup, TigerCS answers 404 ("No interaction exists for this conversation"). Do not send updates for unanswered calls, or treat that 404 as expected.

6. Digital Channel Flow

Genesys channel	channel value	TigerCS channel code	Notes
Website live chat	WebsiteChat	LIVE_CHAT	Send <code>departmentId</code> or <code>departmentCode</code> if the customer chose a department in the chat form.
Website chatbot / virtual agent	WebsiteChat	LIVE_CHAT	The bot is a participant, not a channel. Bot lines use sender <code>VirtualAgent</code> .
WhatsApp	WhatsApp	WHATSAPP	
Social media (e.g. Facebook, Instagram DMs)	SocialMedia	SOCIAL_DM	<code>customerPhone</code> may be absent; that is allowed.



Figure 4 — Digital channel flow.

1. **Conversation starts.** Call `POST /api/genesys/tickets` with the digital `channel`, the `conversationId`, and the department choice or `queueId`. Store `ticketId`.
2. **During the conversation.** If a human is needed, send `PATCH` with `handoff` (§13). If an agent joins, send `agentId` and `agentName`.
3. **Conversation ends or disconnects**, for any reason: agent ends it, customer closes the browser, connection drops, or timeout. Send `PATCH` with `ended` and the **complete available transcript** (§11).
4. **The ticket continues** under TigerCS workflow. The response echoes `ticketStatus` (typically `Open`) to confirm nothing was closed.

RULE — The ticket is never closed because the interaction ended

Ending a conversation marks the interaction as ended and stores the transcript. It never changes the ticket status, resolution, owner or SLA. A customer who asked for an NOC may have a workflow running for days after the chat window closes.

NOTE — AI safety net

If an ended conversation contains `VirtualAgent` messages, no `HumanAgent` message, no resolved handler and no outstanding handoff, and the ticket is not Closed, TigerCS raises pending human work automatically (`WaitingForAgent`, trigger `AiConnectionLost`). Genesys should still send `handoff.required: true` explicitly when it knows a human is needed.

7. Customer Lookup

Endpoint and full response: §4.1. The lookup reuses the same CRM Buyer, PACT and Tasleeh searches as the TigerCS New Ticket wizard. It is read-only.

Response field	Meaning
<code>found</code>	<code>true</code> when Tiger CRM returned at least one buyer or any external source (PACT, Tasleeh) returned at least one customer.
<code>crmStatus</code>	<code>Found</code> · <code>NotFound</code> · <code>AmbiguousMatch</code> · <code>Failed</code> . <code>crmBuyers</code> is populated only when <code>Found</code> .
<code>crmBuyers[]</code>	Existing customer (<code>customer</code>) with every eligible unit (<code>units[]</code> : project, unit number, floor, lead status, customer type).
<code>externalSources[]</code>	One entry per source: <code>source</code> = <code>Pact</code> or <code>Tasleeh</code> ; <code>status</code> = <code>Found</code> · <code>NotFound</code> · <code>Failed</code> ; <code>customers[]</code> with their <code>units[]</code> .
<code>tickets[]</code>	Up to 10 tickets previously created from this phone number, open ones first: number, status, <code>isOpen</code> , summary, current department, creation time. Not limited to the caller's departments.
<code>openTicketCount</code>	How many of those tickets are not Resolved or Closed.

Behaviour rules

- `found:false` returns **200 OK**, not `404`. An unknown caller is a normal outcome. Continue and create the ticket.
- **A source being down is not an error.** It appears as `status: "Failed"` (or `crmStatus: "Failed"`) inside a `200` response.
- **Lookup failure must never lose the inquiry.** If the lookup call itself fails or times out, Genesys must still call Create Ticket. Create runs its own internal lookup, and any failure there is recorded in the audit trail without blocking ticket creation.
- **No customer is auto-selected or verified.** Choosing and verifying the customer stays an explicit TigerCS agent action.

8. Create Ticket

Endpoint, fields and examples: §4.2.

8.1 Idempotency on conversationId



Figure 5 — Same conversationId, same ticket, never a duplicate.

Outcome	HTTP	Meaning
<code>TicketCreated</code>	201	First accepted request for this conversation; a new ticket was created.
<code>AlreadyIngested</code>	200	This conversation already has a ticket. The same <code>ticketId</code> and <code>ticketNumber</code> are returned and nothing is written.

The guarantee holds under concurrent delivery: a unique database index on the conversation ID means that if two identical requests race, one creates the ticket and the other returns `AlreadyIngested`. The idempotency check runs before anything is written, so retries leave nothing behind.

8.2 What Create does

- Checks the channel is active in the TigerCS channel catalogue (`PHONE`, `LIVE_CHAT`, `WHATSAPP`, `SOCIAL_DM`); otherwise `422`.
- Resolves the department (§8.3).
- Runs the internal customer lookup for enrichment only. It never blocks creation.
- Resolves `agentId` to a TigerCS user, if mapped, and records them as the interaction handler (§12).
- Creates the ticket in status **Open, Unclassified** (§9), with the Genesys context stored on the originating interaction.

8.3 Department resolution

Priority	Source	Rule
1	<code>departmentId</code>	The customer's explicit choice (e.g. website chat: Leasing / Customer Service / Maintenance). Must be an active department; otherwise <code>422</code> . No fallback to the queue.
2	<code>departmentCode</code>	The same explicit choice, as a TigerCS department code (case-insensitive). Unknown or inactive → <code>422</code> ; no fallback.
3	<code>queueId</code>	Genesys Queue → TigerCS Department mapping, configured by a TigerCS administrator (Administration → Genesys routing). Unmapped queue or inactive department → <code>422</code> .
—	none of the above	<code>422</code> "No department could be resolved". There is no default department .

IMPORTANT — Configure queue mappings before UAT

Every Genesys queue that can create TigerCS tickets without an explicit department choice must be mapped in TigerCS. Otherwise those inquiries are rejected with **422**, which is intentional: a configuration gap should be visible rather than silently misrouted.

9. Unclassified Tickets and SLA

A Genesys-created ticket reaches an agent before anyone has read the request, so TigerCS does not guess its classification.

Field	Value at creation	Set by
Category	<code>null</code>	TigerCS agent, on classification
Request Type	<code>null</code>	TigerCS agent, on classification
Priority	<code>null</code>	TigerCS agent, on classification
Department	Resolved per §8.3	Genesys input (explicit choice or queue mapping)
Status	<code>Open</code>	TigerCS workflow
SLA state	<code>NotApplicable</code> (no SLA clock)	Starts at classification

- **No placeholder or default classification is ever created.** The department is only a department; it does not imply a category, request type or priority.
- **The business/resolution SLA clock starts at classification** (TigerCS internal operation `POST /api/tickets/{ticketId}/classification`), from that moment and not backdated. An unclassified ticket carries no SLA period, because SLA policy is selected by priority.
- **First Human Response** is recorded from the transcript: the `sentAtUtc` of the first `HumanAgent` message (write-once). `VirtualAgent`, `System` and `Customer` messages never count, and neither does accepting a handoff.
- Unclassified tickets still appear in queues and in Pending Interactions. Human work never waits for classification.

10. Update Ticket / Conversation

Endpoint and example: §4.3. This is the **only** endpoint through which Genesys changes anything after creation.

10.1 Supported fields

Part	Fields	Effect
Agent information	<code>agentId</code> , <code>agentName</code> (top level)	Recorded on the interaction only if not already set (first agent wins). A mapped <code>agentId</code> also sets the interaction handler (§12).
Conversation end	<code>ended.endedAtUtc</code> (defaults to now), <code>ended.endReason</code> (free text, ≤100 chars)	Marks the interaction ended. End time and reason are write-once.
Transcript	<code>ended.transcript[]</code>	Appends messages not already stored (§11).
Human handoff	<code>handoff.required</code> , <code>agentAvailable</code> , <code>mode</code> , <code>reason</code> , <code>trigger</code> , <code>workItemId</code> , <code>assignedAgentId</code>	Raises, assigns or stands down pending human work (§13).

10.2 Not supported (by design)

The contract has **no field** for ticket status, category, request type, priority, owner/assignee, department, resolution or closure. Unknown JSON properties are ignored. Those fields move only through TigerCS's own operations, with their own authorization, audit and SLA consequences.

10.3 Processing order and validation

1. The conversation must exist (404) and must belong to `{ticketId}` (409). This cross-check prevents a transcript from being attached to the wrong ticket.
2. `handoff` is applied first, so work requested in the same call as the end stays outstanding.
3. `ended` and its transcript are applied next. A malformed transcript is rejected in full with 400 before anything is written.
4. Agent information and interaction ownership are recorded last (apply-if-absent).

10.4 Response

Every 200 response returns `ticketStatus` (proof that nothing closed the ticket), `conversationEnded`, the total `transcriptMessageCount`, and the current `handoffStatus` / `ticketAgentHandoffId` whenever the conversation has, or had, pending human work, even if this update did not touch the handoff.

11. Transcript Handling

RULE — Send the complete available transcript when a digital conversation ends or disconnects

Send it in `ended.transcript` on the `PATCH` for the end event, in conversation order. Include bot, customer, agent and system lines. Voice calls may omit the transcript.

11.1 Message fields

Field	Required	Rules
<code>sender</code>	Yes	Exactly one of <code>Customer</code> , <code>VirtualAgent</code> , <code>HumanAgent</code> , <code>System</code> (case-insensitive). Any other value → 400, nothing stored.
<code>sentAtUtc</code>	Yes	ISO-8601 UTC timestamp of the message.
<code>body</code>	Yes	Message text; must not be empty or whitespace → otherwise 400.
<code>externalMessageId</code>	Strongly recommended	Genesys' own stable message ID, max 64 chars . The primary de-duplication key.
<code>senderId</code>	No	Genesys participant or user ID of the sender, max 64 chars.
<code>senderName</code>	No	Display name, max 200 chars.

Sender value	Use for	SLA effect
<code>Customer</code>	The customer	None
<code>VirtualAgent</code>	Chatbot or virtual-agent replies	None. Also marks the conversation as AI-handled (\$6 safety net).
<code>HumanAgent</code>	A person replying to the customer	The first one records First Human Response.
<code>System</code>	Platform notices ("transferred", automated greeting)	None

11.2 Ordering

Messages are stored in the **order delivered** and are never re-sorted by timestamp, because clocks may not be monotonic and two messages may share a timestamp. Send them in true conversation order.

11.3 Retry and de-duplication

- Resending the same `PATCH` is safe: already-stored messages are skipped and `transcriptMessageCount` does not change.
- A message is considered already stored when its `externalMessageId` matches a stored one. **Without** an ID, the fallback match is identical `sender` + `sentAtUtc` + `body`.
- A retry that carries **more** messages (for example after a truncated first delivery) appends only the missing ones. Always resend the **complete** transcript, never a delta that omits earlier lines.

- End time and end reason are write-once. A redelivered end never moves them, but its new transcript lines are still stored.
- Validation is all-or-nothing: one invalid message rejects the whole update (400) and nothing is written. Fix and resend the whole update.

IMPORTANT — Keep IDs within 64 characters

IDs longer than 64 characters are truncated on storage. A truncated `externalMessageId` no longer matches on retry, which would store duplicates. Genesys must confirm its message ID length (§16).

12. Genesys Agent Mapping and Interaction Ownership

Introduced by migration [20260911105154_AddGenesysAgentMappingAndInteractionOwnership](#).

12.1 How mapping works



Figure 6 — Agent resolution uses the immutable Genesys User ID only.

- **The key is the stable Genesys User ID** (the immutable Genesys Cloud user ID). Agent name and email are **never** used to match. `agentEmail` is recorded for audit only.
- Mapping is an **administrative** act: Tiger IT sets `GenesysUserId` on the TigerCS user beforehand. Each Genesys User ID can map to at most one TigerCS user (unique index). TigerCS never creates users automatically.
- In the current build there is **no administration screen or API** for this mapping; Tiger IT maintains it directly in the database.

12.2 Behaviour by mapping state

Situation	Create / PATCH (with agentId)	Agent Context endpoint
Mapped, user active	Ticket or update accepted. Interaction records <code>handledByUserId</code> = mapped user (if not already set).	200 Resolved , with user ID, roles and departments; handler recorded when <code>conversationId</code> is sent.
Mapped, user deactivated	Accepted. Genesys <code>agentId/agentName</code> stored verbatim; no handler recorded.	403 with code: <code>GENESYS_AGENT_INACTIVE</code> .
Not mapped	Accepted. Genesys <code>agentId/agentName</code> stored verbatim; no handler recorded. Audit notes "NotMapped".	403 with code: <code>GENESYS_AGENT_NOT_MAPPED</code> .
No agent sent	Accepted (e.g. queued chat). Handler stays empty.	400 (<code>genesysUserId</code> required).

Create and PATCH are **lenient**: an agent problem never blocks the inquiry. Agent Context is **strict**: it refuses unmapped or inactive agents.

12.3 Stored fields

Stored on the interaction	Meaning	Write rule
<code>GenesysAgentId</code> , <code>GenesysAgentName</code>	The Genesys agent ID and name exactly as sent	First value wins
<code>GenesysAgentUserId</code>	The Genesys User ID that resolved to a TigerCS user	Written together with the handler
<code>HandledByUserId</code>	The TigerCS user who handled this interaction ("Handled By")	First resolved agent wins; later agents never overwrite it

12.4 Handled By is not the ticket owner

Interaction Handled By records who spoke to the customer on one conversation. **Ticket owner** (`CurrentOwnerEmployeeId`) records who is responsible for resolving the business case. They are independent: no Genesys endpoint changes the ticket owner.

Step	Event	Interaction Handled By	Ticket owner
1	Genesys agent Noor (mapped) answers; Create is called with Noor's Genesys User ID	Noor	Per TigerCS assignment rules
2	The ticket is transferred to Collections and assigned to Ahmed in TigerCS	Noor (unchanged)	Ahmed
3	Another Genesys agent later sends an update with a different <code>agentId</code>	Noor (first handler kept)	Ahmed

The only place ownership changes because of an interaction is inside TigerCS: when a TigerCS agent **starts** an item from Pending Interactions (§13.6), they become the ticket owner.

13. Human Handoff

Pending human work applies to **every channel**. It is not a "callback" concept: a callback is one possible continuation mode. The work is attached to the conversation's existing ticket; **a handoff never creates a second ticket**.

13.1 Requesting a human

Request — AI escalates to a human

```
PATCH <TIGERCS_BASE_URL>/api/genesys/tickets/1234
{
  "conversationId": "<GENESYS_CONVERSATION_ID>",
  "agentId": "<GENESYS_USER_ID of the human taking over, if one is available>",
  "handoff": {
    "required": true,
    "agentAvailable": true,
    "mode": "HumanTakeover",
    "trigger": "AiEscalated",
    "reason": "Customer asked about NOC fees",
    "workItemId": "<GENESYS_ROUTING_TASK_ID, if any>"
  }
}
```

13.2 Outcomes

Request	Resulting status	Visible in Pending Interactions?
<code>required:true, agentAvailable:true</code> and top-level <code>agentId</code> sent	Assigned (Genesys agent recorded)	Yes, as assigned
<code>required:true, agentAvailable:true</code> , no <code>agentId</code>	WaitingForAgent	Yes
<code>required:true, agentAvailable:false</code>	WaitingForAgent	Yes: the agent work list
Later: <code>handoff.assignedAgentId</code> = Genesys agent who took it	WaitingForAgent → Assigned	Yes, as assigned
<code>required:false</code> + <code>reason</code> (AI resumed / no longer needed)	Cancelled ; ticket untouched	No (resolved)
<code>handoff.required</code> omitted	No request or cancellation (e.g. an assignment-only update)	Unchanged

NOTE — Two agent fields

The top-level `agentId` names the agent handling the conversation (and, with `agentAvailable:true`, the agent it is handed to). `handoff.assignedAgentId` reports that a Genesys agent took **already-outstanding** work; it requires an open work item (otherwise 422). The assigned Genesys ID is stored as-is; the TigerCS employee on the work item stays empty until a TigerCS agent starts it.

13.3 Supported values

Field	Allowed values	Notes
mode	Callback · ContinueChat · ReplyInChannel · HumanTakeover	Optional. Omit unless Genesys actually knows it; never inferred from the channel. Unknown → 400.
trigger	CustomerRequestedHuman · AiConnectionLost · AiEscalated · RoutingDecision · AgentTransfer	Optional. Omit unless stated. Unknown value: see §14 note.
reason	free text	Why a human is needed. Required when <code>required:false</code> .
workItemId	Genesys routing task ID	Optional; the stronger idempotency key when available.

13.4 Statuses

Status	Meaning	Set by
WaitingForAgent	A human is needed and nobody has taken it	Genesys (or the TigerCS AI safety net)
Assigned	An agent is named but work has not started	Genesys routing, or TigerCS
InProgress	A TigerCS agent is actively handling it	TigerCS agent (Start)
Completed	The human work is done; the ticket is not closed	TigerCS agent (Complete)
Cancelled	The human work is no longer needed	Genesys (<code>required:false</code>) or TigerCS agent

13.5 Idempotency

- At most **one outstanding** work item per conversation. A repeated `required:true` returns that same item (enforced by a filtered unique index, so concurrent requests are safe).
- When `workItemId` is supplied it is checked first.
- A repeated assignment of the same agent changes nothing. A stand-down when nothing is outstanding still returns 200.

13.6 Pending Interactions (TigerCS)

- The TigerCS agent work list of outstanding human work across all channels (website chats a bot could not finish, WhatsApp threads, social messages, callbacks and so on), longest wait first, scoped to the agent's visible departments.
- A TigerCS agent **Starts** an item (it becomes theirs, they become ticket owner, and an Open ticket moves to In Progress), then **Completes** or **Cancels** it. Completing never closes the ticket.
- TigerCS has no "call" or "reply" button. Genesys still performs any callback or message delivery.
- Ending the conversation does **not** resolve outstanding work. The update response keeps reporting `handoffStatus` so Genesys can see it is still pending.

14. Error Handling

Errors use RFC 7807 ProblemDetails (`application/problem+json`). Branch on the HTTP status and, where present, the `code` member. `title` and `detail` are human-readable and may change.

Example error bodies

```
422 Unprocessable Entity
{
  "type": "https://tigercs.internal/problems/genesys-department-not-resolved",
  "title": "No department could be resolved for this inquiry",
  "status": 422,
  "detail": "Genesys queue '<QUEUE_ID>' has no active department mapping."
}

403 Forbidden (agent-context only)
{ "type": "https://tigercs.internal/problems/genesys-agent-not-mapped",
  "title": "Genesys agent is not mapped to a Ticketing user", "status": 403,
  "detail": "Genesys agent is not mapped to a Ticketing user.",
  "code": "GENESYS_AGENT_NOT_MAPPED" }
```

Status	When it occurs	Endpoints	Genesys action
200	Success; also <code>AlreadyIngested</code> , and lookup <code>found:false</code>	All	Continue
201	Ticket created (<code>TicketCreated</code>)	POST tickets	Store <code>ticketId</code>
400	Unsupported <code>channel</code> ; missing/blank <code>conversationId</code> ; missing <code>phoneNumber</code> ; invalid transcript (unknown sender, empty body); unknown handoff <code>mode</code> ; missing/too long <code>genesysUserId</code> ; malformed JSON	All	Fix the payload. Do not retry unchanged.
401	No, invalid or expired JWT	All	Log in again once, then retry
403	Service account lacks CS Agent / CS Supervisor role; or (agent-context) <code>GENESYS_AGENT_NOT_MAPPED</code> / <code>GENESYS_AGENT_INACTIVE</code>	All / agent-context	Fix configuration or mapping. Do not retry.
404	Ticket not found; conversation never ingested (e.g. unanswered call)	PATCH, agent-context	Check IDs. Expected for unanswered calls.
409	<code>conversationId</code> belongs to a different ticket than <code>{ticketId}</code>	PATCH	Use the <code>ticketId</code> returned by Create
422	Department not resolved; channel not configured or inactive in TigerCS; ticket creation refused; <code>assignedAgentId</code> with no outstanding work	POST tickets, PATCH	Configuration or business issue. Alert Tiger IT; do not loop.
503	<code>Genesys:Enabled</code> is false	All	Retry later with back-off; alert Tiger IT

IMPORTANT — Known issue in the current build

On `PATCH`, an unrecognised `handoff.trigger`, or `handoff.required:false` sent **without** a `reason`, currently returns HTTP **500** instead of **400**. Nothing is written. Until this is corrected, treat it as a validation error: correct the payload rather than retrying.

Retry guidance

- Safe to retry at any time: all four endpoints are idempotent for identical input.
- Retry with exponential back-off on network errors, timeouts, **503** and **5xx**. Do **not** retry **400**, **403**, **409** or **422** unchanged.
- If Create cannot be completed after retries, keep the `conversationId`: a later successful Create still yields exactly one ticket.

15. UAT Testing Scenarios

Run each scenario against the UAT environment with `Genesys:Enabled = true`, the service account, queue mappings and agent mappings configured.

ID	Scenario	Steps	Expected result
A	Phone pickup, known customer	Agent answers → lookup with a known number → Create (Phone, queueId, mapped agentId)	Lookup 200 found:true with buyer, units and tickets. Create 201 TicketCreated. Ticket Open, Unclassified, handler = mapped agent.
B	Phone pickup, unknown customer	Lookup with an unknown number → Create	Lookup 200 found:false. Create 201; ticket created as an unverified inquiry.
C	Duplicate Create	Send the same Create twice (also in parallel)	First 201 TicketCreated, then 200 AlreadyIngested with the same ticketId. Only one ticket exists.
D	Website chatbot conversation	Create with WebsiteChat and departmentCode (or departmentId) from the chat form	201; ticket in the chosen department; category, request type and priority empty.
E	Full transcript submission	PATCH with ended + transcript containing Customer, VirtualAgent, HumanAgent, System lines	200 Applied, conversationEnded:true, transcriptMessageCount = N, ticketStatus unchanged (e.g. Open). Transcript visible on the ticket.
F	Transcript retry / de-duplication	Resend E unchanged; then resend with 2 extra messages	Count stays N; then becomes N+2. No duplicate lines. End time unchanged.
G	AI requests human, agent available	PATCH handoff{required:true, agentAvailable:true, mode:"HumanTakeover", trigger:"AiEscalated"} with top-level agentId	200, handoffStatus:"Assigned", ticketAgentHandoffId returned. Same ticket.
H	AI requests human, none available	PATCH handoff{required:true, agentAvailable:false, reason:"..."}	handoffStatus:"WaitingForAgent"; item appears in TigerCS Pending Interactions for the ticket's department.
I	Genesys agent mapping	Agent-context with a mapped genesysUserId and the conversationId	200 Resolved; userId, roles, departments; handledByUserId = mapped user.
J	Unmapped Genesys agent	Agent-context with an unmapped ID; then Create with that ID as agentId	Agent-context 403 GENESYS_AGENT_NOT_MAPPED. Create still 201; agentId stored verbatim; no handler.
K	Conversation ends while human work pending	After H, PATCH with ended + transcript. Variant: AI-only chat ended with no handoff sent.	200, conversationEnded:true, handoffStatus still WaitingForAgent, ticket not closed. Variant: TigerCS auto-raises WaitingForAgent (AiConnectionLost).

Additional negative checks recommended: unsupported channel → 400; unmapped queue without department → 422; PATCH with another ticket's conversationId → 409; request without a token → 401.

16. Information Required from Genesys

#	Item	Why TigerCS needs it
1	Stable Queue IDs for every queue that can create TigerCS tickets, with the intended TigerCS department	Queue → Department mapping (§8.3); unmapped queues are rejected with 422
2	Stable Agent / User IDs (Genesys Cloud user ID) for every agent who will handle TigerCS interactions	Agent mapping and Handled By (§12)
3	Stable Conversation ID : confirm format, ≤64 chars, and that it stays the same across transfers and bot-to-human handover	Idempotency key, one ticket per conversation
4	Stable transcript message ID per message, ≤64 chars	Transcript de-duplication (§11.3)
5	Work item / routing task ID , if available	Stronger handoff idempotency (<code>workItemId</code>)
6	Bot / virtual-agent sender identification : how Genesys marks bot vs human vs system lines	Correct <code>sender</code> values; First Human Response SLA; AI safety net
7	How the website form department choice will be sent (<code>departmentId</code> or <code>departmentCode</code>)	Explicit department selection (§8.3)
8	Which Genesys component will call TigerCS (Data Actions, middleware) and where it will store credentials	Authentication decision (§3.3)
9	When Genesys knows a human is needed, whether an agent is available, and the <code>mode/trigger</code> it can state	Human handoff (§13)
10	Typical <code>endReason</code> values	Reporting (free text is accepted)
11	Sample UAT payloads/events for phone, website chat, chatbot, WhatsApp and social	End-to-end UAT (§15)

17. Security Notes

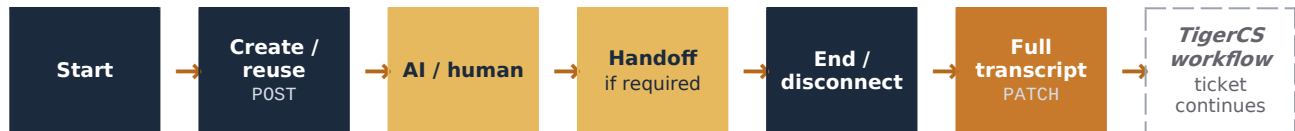
- This document contains **no** production or UAT passwords, API keys, JWTs, signing keys or connection strings. All such values are placeholders in angle brackets.
- Credentials (service account, and in future the integration-layer credential) are **exchanged separately through a secure channel** agreed between Tiger IT and Genesys, never by email, chat, tickets or this document.
- No secret may be committed to Git in any repository, or written into Genesys flows, scripts, logs or architect exports.
- Use a **dedicated** TigerCS service account for the integration with the minimum role (CS Agent). Never use a personal or System Administrator account.
- Only HTTPS endpoints are to be used for UAT and production.
- Do not log **Authorization** headers or full transcripts in Genesys-side diagnostics.
- Rotate the service-account password (and future keys) on a schedule agreed with Tiger IT, and immediately if exposure is suspected.

18. End-to-End Flow Summary

Phone



Digital (website chat, chatbot, WhatsApp, social)



Always	Never
Use one <code>conversationId</code> per conversation and reuse it on every call	Create a ticket on ringing
Store and reuse the <code>ticketId</code> from Create	Expect TigerCS to close a ticket when a conversation ends
Send the complete transcript on end/disconnect	Send category, priority, status or owner
Send the immutable Genesys User ID as <code>agentId</code>	Use agent names or emails as identifiers
Proceed to Create even if the lookup fails	Put credentials in flows, scripts or documents